

Laboratory Manual For

Machine Learning (U24AIMLPC508)

Semester – V

Artificial Intelligence Machine Learning



D. Y. Patil University, Pune, Ambi

CERTIFICATE

This is to certify that **Mr. Akash Kumar Saw,** Roll No **03** and Examination seat no **SOE24BTAM03** of class **Third Year B.Tech,** Branch **Computer Science and Engineering (Artificial Intelligence and Machine Learning)** has Completed all the Practical satisfactorily in Subject **Machine Learning Lab (U24AIMLPC508)** during the semester **V** For the academic year **2026** to **2027** as prescribed in the curriculum.

Date: …….

Course Teacher **Head of The Department**

List of Practical and Assessment

Sr. No.	Name of Experiments	Teacher Sign
1	Write a Program to find the mean, median, standard deviation and mode using user defined functions.	
2	Create a data frame with columns at least 5 observations. a) Retrieve a particular column from the DataFrame b) Summarize the data frame and observe the statistics of the DataFrame created c) Observe the mean and standard deviation of the data frame and print the values.	
3	Write a program to implement the Linear Regression for a sample training data set stored as a .CSV file. Compute Mean Square Error by considering few test data sets.	
4	Write a program to implement the Non-linear Regression for a sample training data set stored as a .CSV file. Compute Mean Square Error by considering few test data sets.	
5	Write a program to implement the Logistic Regression for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier.	
6	Write a program to implement the naive Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.	
7	Write a program to implement k-Nearest Neighbor algorithm to classify the Iris data set. Print both correct and wrong predictions.	
8	Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.	
9	Write a program to demonstrate the working of the decision tree based CART algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.	
10	Implement a single neural network and test for different logic gates.	

PRACTICAL NO. 1

Title: Write a Program to find the mean, median, standard deviation and mode using user defined functions.

Aim

To write a Python program that computes the mean, median, standard deviation, and mode of a numeric dataset using user-defined functions, without relying on built-in statistics libraries.

Objectives

- Implement standard descriptive-statistics measures from first principles.
- Distinguish between measures of central tendency (mean, median, mode) and measures of dispersion (standard deviation).
- Handle both odd- and even-length datasets when computing the median.
- Handle multi-modal datasets when computing the mode.

Theory

Descriptive statistics summarise the main features of a dataset. The mean is the arithmetic average of all values. The median is the middle value of a sorted dataset; for an even number of observations it is the average of the two central values. The standard deviation measures the spread of data around the mean and is computed as the square root of the variance, the average of the squared deviations from the mean. The mode is the value (or values) that occur most frequently in the dataset; a dataset can have more than one mode when several values share the highest frequency.

Concepts Used

- Central tendency (mean, median, mode)
- Dispersion (variance and standard deviation)
- Sorting for median computation
- Frequency counting using Counter

System / Software Requirements

- Python 3.x
- Standard library module: collections
- Text editor / IDE

Algorithm

1. Accept a list of numeric values from the user.

2. Compute the mean by dividing the sum of values by the count of values.

3. Sort the data and compute the median from the middle element(s).

4. Compute the variance as the average squared deviation from the mean, then take its square root to get the standard deviation.

5. Use a frequency counter to find the value(s) with the highest occurrence count as the mode.

6. Print all four computed statistics.

Source Code

descriptive_stats.py

```python
from collections import Counter

def mean(data):
    return sum(data) / len(data)

def median(data):
    s = sorted(data)
    n = len(s)
    if n % 2 == 1:
        return s[n // 2]
    return (s[n // 2 - 1] + s[n // 2]) / 2

def standard_deviation(data):
    m = mean(data)
    variance = sum((x - m) ** 2 for x in data) / len(data)
    return variance ** 0.5

def mode(data):
    counts = Counter(data)
    max_count = max(counts.values())
    return [x for x, count in counts.items() if count == max_count]

data = list(map(float, input("Enter numbers separated by space: ").split()))

print("Mean:", mean(data))
print("Median:", median(data))
print("Standard Deviation:", standard_deviation(data))
print("Mode:", mode(data))
```

Execution

Command

```
python3 descriptive_stats.py
```

Sample Input

Input

```
Enter numbers separated by space: 12 15 12 18 20 15 12 22
```

Sample Output

Output

```
Mean: 15.75
Median: 15.0
Standard Deviation: 3.5637...
Mode: [12.0]
```

Test Cases

Case	Input / Condition	Expected Result
Odd count	5 7 9	median = 7
Even count	4 6 8 10	median = 7.0
Single mode	1 2 2 3	mode = [2]
Multi-modal	1 1 2 2 3	mode = [1, 2]

Result

Thus, a Python program was implemented to compute the mean, median, standard deviation, and mode of a dataset using user-defined functions.

PRACTICAL NO. 2

Title: Create a data frame with columns at least 5 observations. a) Retrieve a particular column from the DataFrame b) Summarize the data frame and observe the statistics of the DataFrame created c) Observe the mean and standard deviation of the data frame and print the values.

Aim

To create a pandas DataFrame with at least five observations and demonstrate column retrieval, statistical summarisation, and computation of mean and standard deviation.

Objectives

- Construct a DataFrame from a Python dictionary of columns.
- Retrieve an individual column as a pandas Series.
- Use describe() to obtain summary statistics of numeric columns.
- Compute the mean and standard deviation of numeric columns directly.

Theory

A pandas DataFrame is a two-dimensional, labelled tabular data structure with columns that may hold different data types. It is the primary data structure used for data analysis in Python. Individual columns can be accessed like dictionary keys and returned as a Series. The describe() method computes count, mean, standard deviation, minimum, quartiles, and maximum for every numeric column in a single call, which is useful for a quick exploratory overview of a dataset before further analysis.

Concepts Used

- DataFrame construction
- Column indexing
- Descriptive summary via describe()
- Column-wise mean and standard deviation

System / Software Requirements

- Python 3.x
- pandas library
- Text editor / IDE

Algorithm

1. Define a dictionary containing at least five observations across several columns.

2. Construct a DataFrame from the dictionary.

3. Retrieve and display a chosen column using DataFrame indexing.

4. Call describe() to display summary statistics of the DataFrame.

5. Compute and display the mean and standard deviation of the numeric columns.

Source Code

dataframe_summary.py

```
import pandas as pd

data = {
    "Name": ["Asha", "Bharat", "Charan", "Divya", "Esha"],
    "Age": [20, 21, 19, 22, 20],
    "Marks": [78, 85, 69, 92, 88]
}

df = pd.DataFrame(data)

print("DataFrame:\n", df)
print("\nMarks column:\n", df["Marks"])
print("\nSummary Statistics:\n", df.describe())
print("\nMean:\n", df.mean(numeric_only=True))
print("\nStandard Deviation:\n", df.std(numeric_only=True))
```

Execution

Command

```
python3 dataframe_summary.py
```

Sample Input

Input

```
(No user input – dataset is defined in code)
```

Sample Output

Output

```
DataFrame:
     Name  Age  Marks
0    Asha   20     78
1  Bharat   21     85
...
Marks column: 0  78 ... 4  88
Summary Statistics: count, mean, std, min, 25%, 50%, 75%, max for Age and Marks
Mean:  Age  20.4  Marks  82.4
Standard Deviation:  Age  1.14  Marks  9.15
```

Test Cases

Case	Input / Condition	Expected Result
Column retrieval	df["Marks"]	returns Marks Series of length 5
Summary	df.describe()	returns 8-row statistics table
Mean	df.mean(numeric_only=True)	returns per-column mean

Case	Input / Condition	Expected Result
Std deviation	df.std(numeric_only=True)	returns per-column std

Result

Thus, a pandas DataFrame was created and column retrieval, statistical summarisation, and mean/standard-deviation computation were demonstrated.

PRACTICAL NO. 3

Title: Write a program to implement the Linear Regression for a sample training data set stored as a .CSV file. Compute Mean Square Error by considering few test data sets.

Aim

To implement Linear Regression on a training dataset stored as a CSV file and evaluate the model using Mean Squared Error on a held-out test dataset.

Objectives

- Persist training and test data as CSV files and read them back with pandas.
- Fit a simple linear regression model of the form Y = mX + c.
- Generate predictions for unseen test inputs.
- Evaluate prediction quality using Mean Squared Error (MSE).

Theory

Linear Regression models the relationship between a dependent variable Y and an independent variable X as a straight line, Y = mX + c, where m is the slope and c is the intercept. The model parameters are learned by minimising the sum of squared residuals between predicted and actual values (ordinary least squares). Once fitted, the model can predict Y for new values of X. Mean Squared Error, the average of the squared differences between predicted and actual test values, is a standard metric for regression accuracy — lower MSE indicates a better fit.

Concepts Used

- Simple linear regression
- Ordinary least squares
- Train/test split via separate CSV files
- Mean Squared Error

System / Software Requirements

- Python 3.x
- pandas, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Create training and test data and save each as a .CSV file.

2. Read the CSV files back into DataFrames.

3. Separate features (X) and target (Y) for both training and test sets.

4. Fit a LinearRegression model on the training data.

5. Predict Y for the test set inputs.

6. Compute and print the Mean Squared Error between predicted and actual test values.

Source Code

linear_regression.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

train = pd.DataFrame({
    "X": [1, 2, 3, 4, 5, 6, 7, 8],
    "Y": [3, 5, 7, 9, 11, 13, 15, 17]
})
test = pd.DataFrame({
    "X": [9, 10, 11],
    "Y": [19, 21, 23]
})

train.to_csv("linear_train.csv", index=False)
test.to_csv("linear_test.csv", index=False)

train_data = pd.read_csv("linear_train.csv")
test_data = pd.read_csv("linear_test.csv")

X_train = train_data[["X"]]
y_train = train_data["Y"]
X_test = test_data[["X"]]
y_test = test_data["Y"]

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

mse = mean_squared_error(y_test, predictions)
print("Predictions:", predictions)
print("Mean Squared Error:", mse)
```

Execution

Command

```
python3 linear_regression.py
```

Sample Input

Input

```
linear_train.csv: X=1..8, Y=2X+1  |  linear_test.csv: X=9,10,11
```

Sample Output

Output

```
Predictions: [19. 21. 23.]
Mean Squared Error: 0.0
```

Test Cases

Case	Input / Condition	Expected Result
Perfectly linear data	Y = 2X + 1	MSE ≈ 0
Held-out test points	X = 9,10,11	predictions close to actual Y
Model type	LinearRegression()	fits without error on numeric CSV

Result

Thus, a Linear Regression model was implemented using a CSV-based training and test dataset, and its accuracy was evaluated using Mean Squared Error.

PRACTICAL NO. 4

Title: Write a program to implement the Non-linear Regression for a sample training data set stored as a .CSV file. Compute Mean Square Error by considering few test data sets.

Aim

To implement Non-linear (Polynomial) Regression on a training dataset stored as a CSV file and evaluate the model using Mean Squared Error on a test dataset.

Objectives

- Transform a single feature into polynomial features of a chosen degree.
- Fit a linear model over the expanded polynomial feature space to capture non-linear relationships.
- Generate predictions for unseen test inputs.
- Evaluate prediction quality using Mean Squared Error.

Theory

Many real-world relationships are not linear. Polynomial regression addresses this by expanding a single feature X into higher-degree terms (X, X^2, X^3, ...) and fitting an ordinary linear regression over this expanded feature space. Although the model is still linear in its parameters, the resulting curve, $Y = a + bX + cX^2 + ...$, can fit non-linear (curved) data. The degree of the polynomial controls the flexibility of the fit; scikit-learn's PolynomialFeatures generates the expanded feature matrix automatically.

Concepts Used

- Polynomial feature expansion
- Non-linear curve fitting via linear regression
- Degree selection
- Mean Squared Error

System / Software Requirements

- Python 3.x
- pandas, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Create training and test data reflecting a quadratic relationship and save each as a .CSV file.

2. Read the CSV files back into DataFrames.

3. Expand the X feature into polynomial terms of degree 2 using PolynomialFeatures.

4. Fit a LinearRegression model on the expanded training features.

5. Transform the test features the same way and predict Y.

6. Compute and print the Mean Squared Error between predicted and actual test values.

Source Code

nonlinear_regression.py

```
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

train = pd.DataFrame({
    "X": [1, 2, 3, 4, 5, 6, 7, 8],
    "Y": [3, 6, 11, 18, 27, 38, 51, 66]
})
test = pd.DataFrame({
    "X": [9, 10, 11],
    "Y": [83, 102, 123]
})

train.to_csv("nonlinear_train.csv", index=False)
test.to_csv("nonlinear_test.csv", index=False)

train_data = pd.read_csv("nonlinear_train.csv")
test_data = pd.read_csv("nonlinear_test.csv")

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train = poly.fit_transform(train_data[["X"]])
X_test = poly.transform(test_data[["X"]])

model = LinearRegression()
model.fit(X_train, train_data["Y"])
predictions = model.predict(X_test)

mse = mean_squared_error(test_data["Y"], predictions)
print("Predictions:", predictions)
print("Mean Squared Error:", mse)
```

Execution

Command
python3 nonlinear_regression.py

Sample Input

Input
nonlinear_train.csv: X=1..8, Y=X^2+X+1 | nonlinear_test.csv: X=9,10,11

Sample Output

Output
Predictions: [83. 102. 123.] Mean Squared Error: ~0.0

Test Cases

Case	Input / Condition	Expected Result
Quadratic data	$Y = X^2 + X + 1$	MSE ≈ 0 with degree=2
Degree too low	degree=1 on quadratic data	noticeably higher MSE
Held-out points	X = 9,10,11	predictions close to actual Y

Result

Thus, a Non-linear (Polynomial) Regression model was implemented using a CSV-based training and test dataset, and its accuracy was evaluated using Mean Squared Error.

PRACTICAL NO. 5

Title: Write a program to implement the Logistic Regression for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier.

Aim

To implement Logistic Regression on a sample dataset stored as a CSV file and compute the classification accuracy on a held-out test split.

Objectives

- Understand logistic regression as a classification technique for binary outcomes.
- Split a dataset into stratified training and test partitions.
- Fit a LogisticRegression classifier and generate class predictions.
- Evaluate the classifier using accuracy score.

Theory

Logistic Regression is a classification algorithm used to predict a binary outcome. Instead of fitting a straight line, it fits a sigmoid (logistic) function that maps any real-valued input to a probability between 0 and 1. A threshold (typically 0.5) converts this probability into a class label. Unlike linear regression, logistic regression is trained by maximising the likelihood of the observed labels rather than minimising squared error. Accuracy, the proportion of correctly classified test samples, is a standard evaluation metric for balanced classification problems.

Concepts Used

- Binary classification
- Sigmoid function
- Stratified train/test split
- Accuracy score

System / Software Requirements

- Python 3.x
- pandas, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Create a sample dataset relating study hours to pass/fail outcome and save it as a .CSV file.

2. Read the CSV file back into a DataFrame.

3. Split the data into training and test sets using a stratified split to preserve class balance.

4. Fit a LogisticRegression model on the training data.

5. Predict class labels for the test set.

6. Compute and print the classification accuracy.

Source Code

logistic_regression.py

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

sample = pd.DataFrame({
    "Hours": [1, 2, 2.5, 3, 3.5, 4, 5, 5.5, 6, 7, 8, 9],
    "Pass": [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1]
})
sample.to_csv("logistic_data.csv", index=False)

data = pd.read_csv("logistic_data.csv")
X = data[["Hours"]]
y = data["Pass"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42, stratify=y
)

model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Accuracy:", accuracy_score(y_test, predictions))
```

Execution

Command

```
python3 logistic_regression.py
```

Sample Input

Input

```
logistic_data.csv: Hours vs Pass (0/1), 12 rows
```

Sample Output

Output

```
Predictions: [0 1 1 1]
Accuracy: 1.0
```

Test Cases

Case	Input / Condition	Expected Result
Well-separated classes	Hours < 3 → 0, Hours ≥ 3.5 → 1	accuracy = 1.0
Stratified split	test_size=0.33	both classes present in test set
Borderline input	Hours = 3.5	predicted class = 1

Result

Thus, a Logistic Regression classifier was implemented on a CSV-based dataset and its accuracy was evaluated on a held-out test split.

PRACTICAL NO. 6

Title: Write a program to implement the naive Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

Aim

To implement a Gaussian Naive Bayes classifier on a sample dataset stored as a CSV file and compute its accuracy on a held-out test split.

Objectives

- Understand the Naive Bayes assumption of conditional feature independence.
- Fit a Gaussian Naive Bayes classifier on multi-feature numeric data.
- Generate predictions for a held-out test split.
- Evaluate the classifier using accuracy score.

Theory

The Naive Bayes classifier applies Bayes' theorem with the (naive) assumption that features are conditionally independent given the class label. For continuous features, the Gaussian Naive Bayes variant models each feature within each class as a normal distribution and estimates the class posterior probability from the product of per-feature likelihoods and the class prior. Despite its simplifying independence assumption, Naive Bayes performs well in practice and is computationally efficient, making it a popular baseline classifier.

Concepts Used

- Bayes' theorem
- Conditional independence assumption
- Gaussian likelihood estimation
- Accuracy score

System / Software Requirements

- Python 3.x
- pandas, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Create a sample two-feature dataset with class labels and save it as a .CSV file.

2. Read the CSV file back into a DataFrame and separate features and target.

3. Split the data into stratified training and test sets.

4. Fit a GaussianNB classifier on the training data.

5. Predict class labels for the test set.

6. Compute and print the classification accuracy.

Source Code

naive_bayes.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

sample = pd.DataFrame({
    "Feature1": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
    "Feature2": [2, 1, 3, 2, 4, 5, 6, 5, 7, 8, 9, 10],
    "Class": [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
})
sample.to_csv("naive_bayes_data.csv", index=False)

data = pd.read_csv("naive_bayes_data.csv")
X = data[["Feature1", "Feature2"]]
y = data["Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42, stratify=y
)

model = GaussianNB()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Accuracy:", accuracy_score(y_test, predictions))
```

Execution

Command

```
python3 naive_bayes.py
```

Sample Input

Input

```
naive_bayes_data.csv: Feature1, Feature2, Class (0/1), 12 rows
```

Sample Output

Output

```
Predictions: [0 1 1 1]
Accuracy: 1.0
```

Test Cases

Case	Input / Condition	Expected Result
Well-separated classes	low feature values → 0, high → 1	accuracy = 1.0
Stratified split	test_size=0.33	both classes represented in test set
Two-feature input	Feature1, Feature2	GaussianNB fits without error

Result

Thus, a Gaussian Naive Bayes classifier was implemented on a CSV-based dataset and its accuracy was evaluated on a held-out test split.

PRACTICAL NO. 7

Title: Write a program to implement k-Nearest Neighbor algorithm to classify the Iris data set. Print both correct and wrong predictions.

Aim

To implement the k-Nearest Neighbor (k-NN) algorithm to classify the Iris dataset and print both correct and incorrect predictions.

Objectives

- Load and split the standard Iris dataset for classification.
- Fit a k-NN classifier with a chosen number of neighbours.
- Compare predicted labels against actual labels for each test sample.
- Report which predictions were correct and which were wrong.

Theory

The k-Nearest Neighbor algorithm is a non-parametric, instance-based classification method. To classify a new sample, it identifies the k closest training samples (typically by Euclidean distance) and assigns the class that is most common among those neighbours. k-NN requires no explicit training phase beyond storing the data, making it simple to implement, though prediction cost grows with dataset size. The Iris dataset, containing sepal and petal measurements for three species, is a standard benchmark for evaluating classification algorithms.

Concepts Used

- Instance-based learning
- Euclidean distance and majority voting
- Choice of k
- Iris benchmark dataset

System / Software Requirements

- Python 3.x
- scikit-learn library
- Text editor / IDE

Algorithm

1. Load the Iris dataset using scikit-learn's built-in loader.

2. Split the data into training and test sets.

3. Fit a KNeighborsClassifier with k = 3 on the training data.

4. Predict the class for every test sample.

5. Compare each prediction with the actual label and print the result labelled Correct or Wrong.

Source Code

knn_iris.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

for actual, predicted in zip(y_test, predictions):
    status = "Correct" if actual == predicted else "Wrong"
    print(
        "Actual:", iris.target_names[actual],
        "Predicted:", iris.target_names[predicted],
        "-", status
    )
```

Execution

Command

```
python3 knn_iris.py
```

Sample Input

Input

```
Iris dataset (150 samples, 4 features, 3 classes), 30% held out for testing
```

Sample Output

Output

```
Actual: setosa Predicted: setosa - Correct
Actual: versicolor Predicted: versicolor - Correct
Actual: virginica Predicted: versicolor - Wrong
... (45 test samples total)
```

Test Cases

Case	Input / Condition	Expected Result
k = 3	n_neighbors=3	high overall accuracy (~95-100%)
Class boundary sample	versicolor/virginica overlap region	occasional Wrong prediction
Well-separated class	setosa samples	consistently Correct

Result

Thus, the k-Nearest Neighbor algorithm was implemented to classify the Iris dataset, and both correct and wrong predictions were printed.

PRACTICAL NO. 8

Title: Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Aim

To demonstrate the ID3 (entropy-based) decision tree algorithm on the Play Tennis dataset and use the trained tree to classify a new sample.

Objectives

- Encode categorical features suitably for a scikit-learn decision tree.
- Build a decision tree using the entropy (information-gain) splitting criterion, consistent with ID3.
- Train the tree on the classic Play Tennis dataset.
- Classify a new, unseen sample using the trained tree.

Theory

ID3 (Iterative Dichotomiser 3) builds a decision tree by recursively selecting, at each node, the attribute that yields the highest information gain — the reduction in entropy (uncertainty about the class label) achieved by splitting on that attribute. This process continues until nodes become pure (all samples share one class) or no attributes remain. Since scikit-learn's DecisionTreeClassifier operates on numeric input, categorical attributes (Outlook, Temperature, Humidity, Wind) are first one-hot encoded, and the tree is configured with criterion="entropy" to match ID3's splitting rule.

Concepts Used

- Entropy and information gain
- One-hot encoding of categorical attributes
- Recursive tree construction
- Classification of a new sample

System / Software Requirements

- Python 3.x
- pandas, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Prepare the Play Tennis dataset with categorical attributes Outlook, Temperature, Humidity, and Wind, and the target Play (Yes/No).

2. One-hot encode the categorical attributes using ColumnTransformer and OneHotEncoder.

3. Build a pipeline combining the encoder with a DecisionTreeClassifier using the entropy criterion.

4. Fit the pipeline on the full dataset.

5. Encode a new sample the same way and predict its class using the trained tree.

Source Code

id3_decision_tree.py

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier

# Play Tennis data
X = pd.DataFrame({
    "Outlook": ["Sunny", "Sunny", "Overcast", "Rain", "Rain", "Rain", "Overcast", "Sunny"],
    "Temperature": ["Hot", "Hot", "Hot", "Mild", "Cool", "Cool", "Cool", "Mild"],
    "Humidity": ["High", "High", "High", "High", "Normal", "Normal", "Normal", "High"],
    "Wind": ["Weak", "Strong", "Weak", "Weak", "Weak", "Strong", "Strong", "Weak"]
})
y = ["No", "No", "Yes", "Yes", "Yes", "No", "Yes", "No"]

preprocessor = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), X.columns)
])

id3 = Pipeline([
    ("prep", preprocessor),
    ("tree", DecisionTreeClassifier(criterion="entropy", random_state=42))
])
id3.fit(X, y)

new_sample = pd.DataFrame([{
    "Outlook": "Overcast",
    "Temperature": "Mild",
    "Humidity": "High",
    "Wind": "Weak"
}])
print("Prediction:", id3.predict(new_sample)[0])
```

Execution

Command
python3 id3_decision_tree.py

Sample Input

Input
Play Tennis dataset (8 rows) + new sample: Overcast, Mild, High, Weak

Sample Output

```
Output
Prediction: Yes
```

Test Cases

Case	Input / Condition	Expected Result
Overcast outlook	Outlook = Overcast	predicted Yes (Overcast is always Yes in training data)
Known training row	Sunny, Hot, High, Weak	predicted matches training label No
Unseen category	handle_unknown="ignore"	pipeline does not error on unseen categories

Result

Thus, the ID3 decision-tree algorithm was demonstrated on the Play Tennis dataset, and a new sample was successfully classified using the trained tree.

PRACTICAL NO. 9

Title: Write a program to demonstrate the working of the decision tree based CART algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Aim

To demonstrate the CART (Gini-index based) decision tree algorithm on an Age/Income dataset and use the trained tree to classify a new sample.

Objectives

- Build a decision tree using the Gini impurity splitting criterion, consistent with CART.
- Train the tree on numeric Age and Income features against a binary class label.
- Classify a new, unseen sample using the trained tree.

Theory

CART (Classification and Regression Trees) builds a binary decision tree by choosing, at each node, the split that minimises Gini impurity — a measure of how often a randomly chosen element would be misclassified if labelled according to the class distribution at that node. Unlike ID3, CART always produces binary splits and can handle both categorical and numeric attributes directly. scikit-learn's DecisionTreeClassifier implements the CART algorithm when configured with criterion="gini" (its default).

Concepts Used

- Gini impurity
- Binary tree splits
- Numeric feature thresholds
- Classification of a new sample

System / Software Requirements

- Python 3.x
- pandas, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Prepare a dataset with numeric Age and Income features and a binary class label.

2. Build a DecisionTreeClassifier configured with the Gini impurity criterion.

3. Fit the tree on the training data.

4. Present a new sample with Age and Income values.

5. Predict and print the class of the new sample.

Source Code

cart_decision_tree.py

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

X = pd.DataFrame({
    "Age": [22, 25, 28, 32, 35, 40, 45, 50],
    "Income": [25000, 30000, 35000, 45000, 50000, 65000, 70000, 85000]
})
y = [0, 0, 0, 1, 1, 1, 1, 1]

model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X, y)

new_sample = pd.DataFrame({
    "Age": [30],
    "Income": [42000]
})
prediction = model.predict(new_sample)[0]
print("Predicted Class:", prediction)
```

Execution

Command
python3 cart_decision_tree.py

Sample Input

Input
Age/Income dataset (8 rows) + new sample: Age=30, Income=42000

Sample Output

Output
Predicted Class: 1

Test Cases

Case	Input / Condition	Expected Result
Low age/income	Age=22, Income=25000	predicted class 0
High age/income	Age=50, Income=85000	predicted class 1
Boundary sample	Age=30, Income=42000	predicted class near decision boundary

Result

Thus, the CART decision-tree algorithm was demonstrated on an Age/Income dataset, and a new sample was successfully classified using the trained tree.

PRACTICAL NO. 10

Title: Implement a single neural network and test for different logic gates.

Aim

To implement a single-hidden-layer neural network and test its ability to learn the AND, OR, and XOR logic gates.

Objectives

- Represent the truth table of a two-input logic gate as a training dataset.
- Train a multi-layer perceptron classifier on each gate's truth table.
- Verify that the trained network reproduces the correct gate outputs.
- Observe the difference between linearly separable gates (AND, OR) and the non-linearly separable XOR gate.

Theory

A neural network learns a mapping from inputs to outputs through weighted connections between layers of nodes, adjusted during training to minimise prediction error. A single-layer perceptron can represent linearly separable functions such as AND and OR, but cannot represent XOR, which is not linearly separable. Adding a hidden layer, as in a multi-layer perceptron (MLP), allows the network to learn non-linear decision boundaries and correctly represent XOR. scikit-learn's MLPClassifier provides a ready-made feed-forward neural network with configurable hidden layers and a choice of optimisation solver.

Concepts Used

- Perceptron and linear separability
- Multi-layer perceptron (MLP)
- Hidden layers for non-linear functions
- AND, OR, XOR truth tables

System / Software Requirements

- Python 3.x
- numpy, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Define the four possible two-bit input combinations (00, 01, 10, 11).

2. Define the expected output for each logic gate (AND, OR, XOR) as a truth table.

3. For each gate, train an MLPClassifier with one hidden layer of 4 neurons on the input/output pairs.

4. Predict the output for all four input combinations using the trained network.

5. Print the predicted outputs alongside the inputs for each gate.

Source Code

neural_network_logic_gates.py

```
import numpy as np
from sklearn.neural_network import MLPClassifier

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

gates = {
    "AND": np.array([0, 0, 0, 1]),
    "OR": np.array([0, 1, 1, 1]),
    "XOR": np.array([0, 1, 1, 0])
}

for gate, y in gates.items():
    model = MLPClassifier(
        hidden_layer_sizes=(4,),
        solver="lbfgs",
        max_iter=5000,
        random_state=42
    )
    model.fit(X, y)
    predictions = model.predict(X)
    print("\n", gate, "Gate")
    for inp, out in zip(X, predictions):
        print(inp, "->", out)
```

Execution

Command

```
python3 neural_network_logic_gates.py
```

Sample Input

Input

```
Fixed truth-table inputs: [0,0], [0,1], [1,0], [1,1]
```

Sample Output

Output

```
AND Gate
[0 0] -> 0   [0 1] -> 0   [1 0] -> 0   [1 1] -> 1
OR Gate
[0 0] -> 0   [0 1] -> 1   [1 0] -> 1   [1 1] -> 1
XOR Gate
```

Output

```
[0 0] -> 0   [0 1] -> 1   [1 0] -> 1   [1 1] -> 0
```

Test Cases

Case	Input / Condition	Expected Result
AND gate	hidden_layer_sizes=(4,)	all 4 outputs match AND truth table
OR gate	hidden_layer_sizes=(4,)	all 4 outputs match OR truth table
XOR gate	hidden_layer_sizes=(4,)	all 4 outputs match XOR truth table (needs hidden layer)

Result

Thus, a single-hidden-layer neural network was implemented and successfully trained to reproduce the AND, OR, and XOR logic gates.